E-Commerce Customer Segmentation

Grouping 4,300+ retail customers into actionable marketing segments with RFM analysis & K-Means clustering

Author	Rahul
Domain	Marketing / Retail Analytics
Dataset	UCI Online Retail — ~541K transactions, UK retailer (2010–11)
Technique	RFM feature engineering + K-Means (unsupervised)
Tech stack	Python · Pandas · NumPy · Scikit-learn · Matplotlib · Seaborn · Power BI
Deliverables	Segment model · customer_segments.csv · 3-page dashboard

Project report — prepared for academic / portfolio submission.

1. Executive Summary

Treating every customer the same wastes marketing budget — a one-off bargain hunter and a high-value repeat buyer should not receive the same email. This project segments 4,338 customers from a UK online retailer using **RFM analysis** (how *recently*, how *frequently*, and how much in *monetary* terms each customer buys) combined with **K-Means clustering**, then translates the resulting clusters into four marketing-ready segments: Champions, Loyal Customers, At Risk and Hibernating.

The analysis confirms a textbook **Pareto pattern**: the 679 Champions are only 15.7% of customers but drive 73.8% of revenue, while the 1,549 Hibernating customers (35.7% of the base) contribute just 2.5%. This directly informs where marketing spend should — and should not — go, and the segments are exported to a CSV that feeds a three-page Power BI marketing dashboard.

4,338	£11.8M	4	73.8%
Customers segmented	Total revenue	Actionable segments	Revenue from Champions

2. Problem Statement & Business Context

Marketing budgets are finite. Spent uniformly across a customer base, they over-invest in people who will never return and under-invest in the loyal few who generate most of the revenue. The goal of this project is to let the business **tailor its strategy per segment**:

- **Identify the high-value core** — who the Champions are, and protect them.
- **Catch churn early** — isolate previously valuable customers whose recency is creeping up, while a win-back is still cheaper than re-acquisition.
- **Stop wasting spend** — recognise low-value dormant customers and cap reactivation cost.
- **Operationalise it** — hand marketing a labelled customer list and a dashboard, not a statistics lecture.

3. Dataset Overview

The project uses the **UCI Online Retail** dataset — roughly 541,000 transaction line items from a UK-based online gift retailer between December 2010 and December 2011. It is transaction-level data (one row per product per invoice), which must be aggregated up to the customer level before any segmentation can happen.

Property	Value
Raw size	~541,000 transaction line items, 8 columns
After cleaning	~400,000+ valid transactions
Customers	4,338 unique customers with a usable ID
Period	Dec 2010 – Dec 2011 (UK retailer, figures in GBP £)
Key fields	InvoiceNo, InvoiceDate, CustomerID, Quantity, UnitPrice, Country
Type of problem	Unsupervised — no target label; structure is discovered, not predicted

This is an unsupervised problem. Unlike a churn classifier, there is no ‘correct’ segment label to learn from. The model finds natural groupings in behaviour, and the analyst is responsible for interpreting and naming them — which makes feature engineering and validation of the clusters especially important.

4. Methodology & Workflow

The pipeline moves from raw transactions to a labelled, dashboard-ready customer list:

5. Data Cleaning

Transaction data is messy. Around 135,000 rows have no CustomerID (anonymous web traffic that cannot be attributed to a person), and the file also contains cancelled orders, returns (negative quantities) and zero-priced rows. All of these are removed before any customer can be scored, because each would distort the RFM totals.

```
# 1. Drop transactions with no CustomerID - cannot attribute to a person
df = df.dropna(subset=["CustomerID"])

# 2. Remove cancellations (InvoiceNo starts with "C"), returns and zero-price rows
df = df[~df["InvoiceNo"].astype(str).str.startswith("C")]
df = df[(df["Quantity"] > 0) & (df["UnitPrice"] > 0)]

# 3. De-duplicate, then compute line-item revenue
df = df.drop_duplicates()
df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]
```

Why each step matters: keeping anonymous rows would mean segmenting a customer who doesn't exist; leaving cancellations and returns in would inflate frequency and corrupt monetary value (a returned order should not count as spend). The cleaning removes roughly a quarter of the rows but leaves ~400K trustworthy transactions across 4,338 customers.

6. RFM Feature Engineering

RFM reduces every customer's entire purchase history to three behavioural numbers that marketers have trusted for decades:

Feature	Definition	Signal
Recency	Days since the customer's last purchase (from a snapshot date)	Lower = more engaged
Frequency	Number of distinct invoices (purchase occasions)	Higher = more loyal
Monetary	Total amount the customer has spent	Higher = more valuable

```
# Snapshot date = day after the last transaction in the data
snapshot_date = df["InvoiceDate"].max() + pd.Timedelta(days=1)

rfm = df.groupby("CustomerID").agg(
    Recency   = ("InvoiceDate", lambda x: (snapshot_date - x.max()).days),
    Frequency = ("InvoiceNo", "nunique"),   # distinct invoices, NOT row count
    Monetary  = ("TotalAmount", "sum"),
).reset_index()
```

A subtle but important choice: Frequency counts *distinct invoices* (`nunique`), not rows. A single shopping trip can contain twenty product lines — counting rows would inflate frequency by basket size and confuse 'buys often' with 'buys a lot at once'. One invoice = one genuine purchase occasion.

7. Handling Skew & Scaling

K-Means measures distance between points, so the raw scale of each feature matters. The RFM values are heavily right-skewed — a handful of 'whale' customers spend orders of magnitude more than the median (£586). Left untreated, those few outliers would dominate the distance calculation and pull cluster centroids toward themselves. The fix is a `log1p` transform to compress the long tail, followed by `StandardScaler` so all three features carry equal weight.

```
rfm_log = rfm[["Recency","Frequency","Monetary"]].apply(np.log1p)  # compress skew
rfm_scaled = StandardScaler().fit_transform(rfm_log)               # equal weighting
```

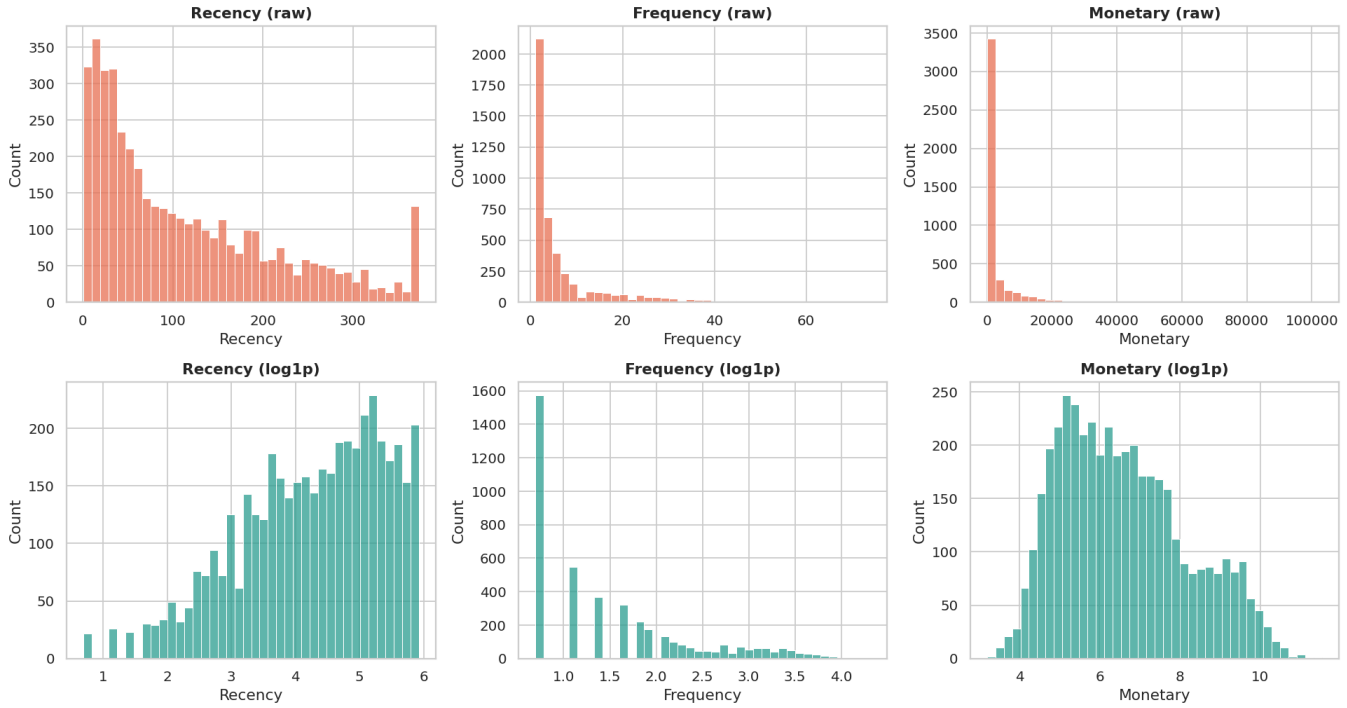

Figure 1 — Each RFM feature before (orange, raw) and after (teal, log1p) transformation. The extreme right-skew collapses into a far more symmetric distribution suitable for distance-based clustering.

8. Choosing the Number of Clusters (k)

K-Means needs k specified up front, so two diagnostics are run across k = 2 to 10: the **elbow method** (within-cluster sum of squares, which always falls as k rises — the 'elbow' marks diminishing returns) and the **silhouette score** (how cleanly separated the clusters are, higher is better).

```
for k in range(2, 11):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(rfm_scaled)
    inertias.append(km.inertia_)
    silhouettes.append(silhouette_score(rfm_scaled, km.labels_))
```

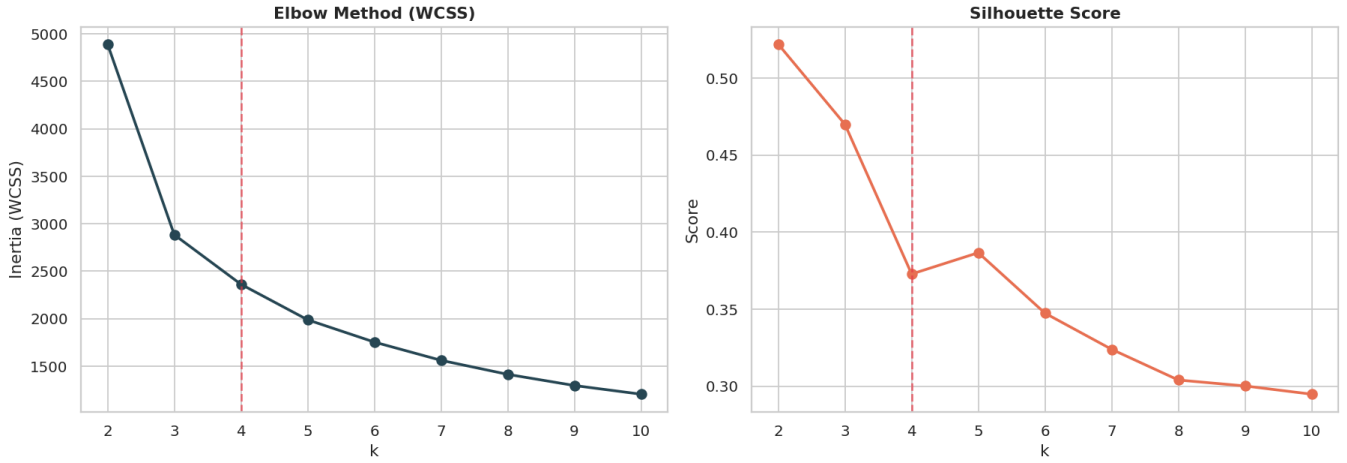

Figure 2 — Elbow (left) and silhouette (right). The dashed line marks the chosen k = 4.

An honest read of these charts. The elbow bends and begins to flatten around k = 3–4. The silhouette score is technically highest at **k = 2** (~0.52) and declines from there — but k = 2 only splits the base into 'engaged' versus 'lapsed', which is too coarse to build a marketing plan on. **k = 4 was chosen as the balance between statistical cohesion and business actionability**: the silhouette at k = 4 (~0.37) still indicates moderate, reasonable separation, the elbow supports it, and four groups map cleanly onto the four standard, operationally distinct RFM personas marketing teams already work with. This trade-off — not blindly maximising a single metric — is the realistic way k is chosen in practice.

9. Clustering & Segment Profiles

The final K-Means model is fit with k = 4. Crucially, clusters are then profiled on the **raw, interpretable** RFM averages (not the scaled values) and named from that profile — because K-Means assigns arbitrary cluster numbers that change between runs, segments are always mapped from the behaviour table, never from a hard-coded cluster ID.

```
kmeans = KMeans(n_clusters=4, n_init=10, random_state=42)
rfm["Cluster"] = kmeans.fit_predict(rfm_scaled)

# Profile on RAW values so the numbers are human-readable, then name the segments
profile = rfm.groupby("Cluster").agg(
    Customers=("CustomerID","count"), AvgRecency=("Recency","mean"),
    AvgFrequency=("Frequency","mean"), AvgMonetary=("Monetary","mean")
).sort_values("AvgMonetary", ascending=False)
```

Segment	Customers	% of base	Avg Recency (days)	Avg Frequency	Avg Monetary (£)	% of revenue
Champions	679	15.7%	15	23.4	12,838	73.8%
Loyal Customers	1,001	23.1%	46	6.8	2,091	17.7%
At Risk	1,109	25.6%	94	2.6	636	6.0%
Hibernating	1,549	35.7%	214	1.2	192	2.5%

The four segments are immediately readable from this table. **Champions** bought ~15 days ago, ~23 times, spending ~£12,800 each. **Hibernating** customers last bought ~214 days ago, just once, for ~£192. The scatter below shows the same separation visually.

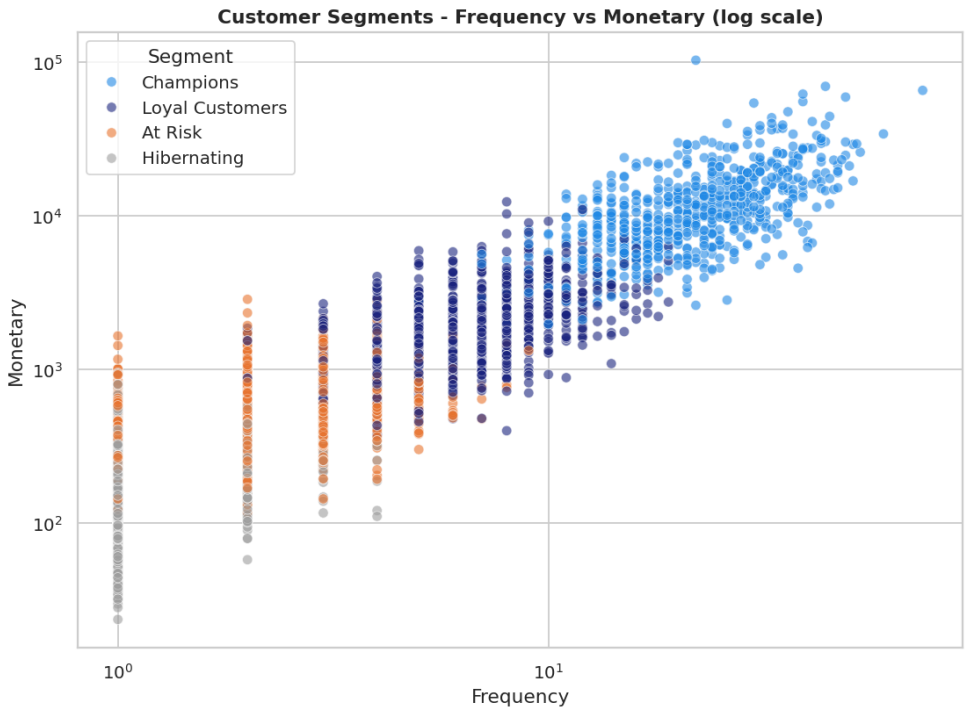

Figure 3 — Frequency vs Monetary (log scale), coloured by segment. Champions (blue) occupy the high-frequency, high-spend corner; Hibernating (grey) sit at the low-frequency, low-spend origin.

10. Segment Size vs Revenue — the Pareto Effect

The most decision-relevant chart in the project contrasts how *many* customers each segment holds against how much *revenue* it generates. The two are almost inverted.

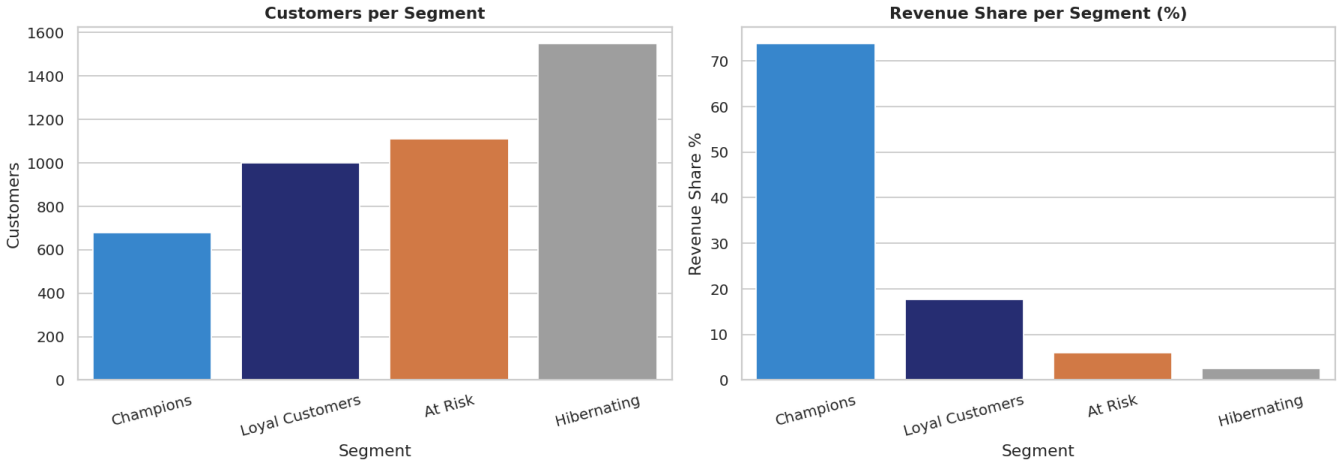

Figure 4 — Customer count (left) vs revenue share (right) per segment.

The headline: Champions are the *smallest* group (15.7% of customers) yet generate **73.8% of all revenue**. At the other end, Hibernating customers are the *largest* group (35.7%) but contribute only 2.5%. This is the classic Pareto / 80-20 pattern, and it is the single most important input to the marketing budget: **the bulk of spend should protect and grow the few high-value segments, not chase the dormant majority.**

11. Power BI Dashboard

The labelled customer list (`customer_segments.csv`) feeds a three-page Power BI dashboard built for a marketing audience — the analytical output turned into something a campaign manager can filter and act on.

Page 1 — Overview

Top-line KPIs (customers, revenue, transactions, segment count) above the revenue split, customers per segment, and average revenue per customer.

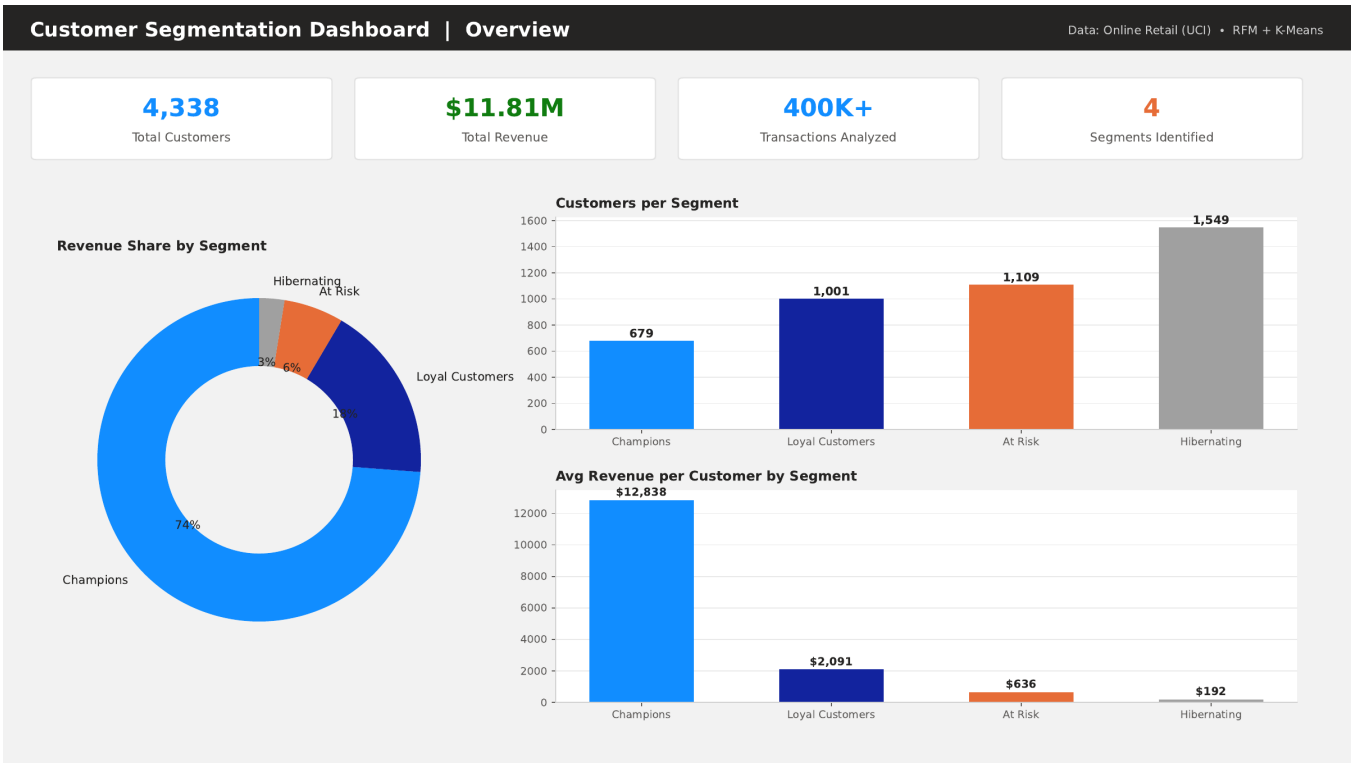

Figure 5 — Dashboard Page 1: Overview.

Page 2 — RFM Segment Profiles

The behavioural fingerprint of each segment: the Frequency-vs-Monetary scatter plus average Recency, Frequency and Monetary side by side.

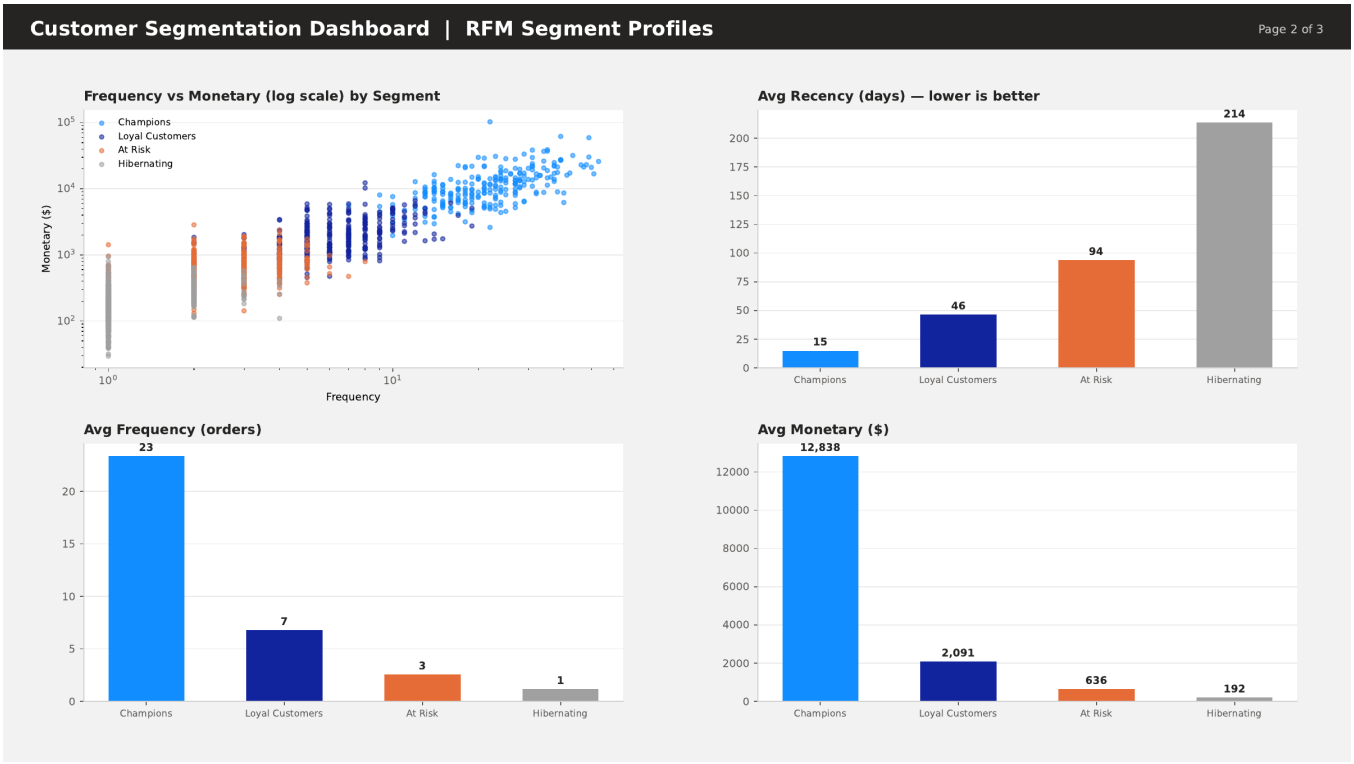

Figure 6 — Dashboard Page 2: RFM segment profiles.

Page 3 — Marketing Action Plan

The payoff page: each segment with its size, revenue share and a concrete recommended action, ready to brief a marketing team.

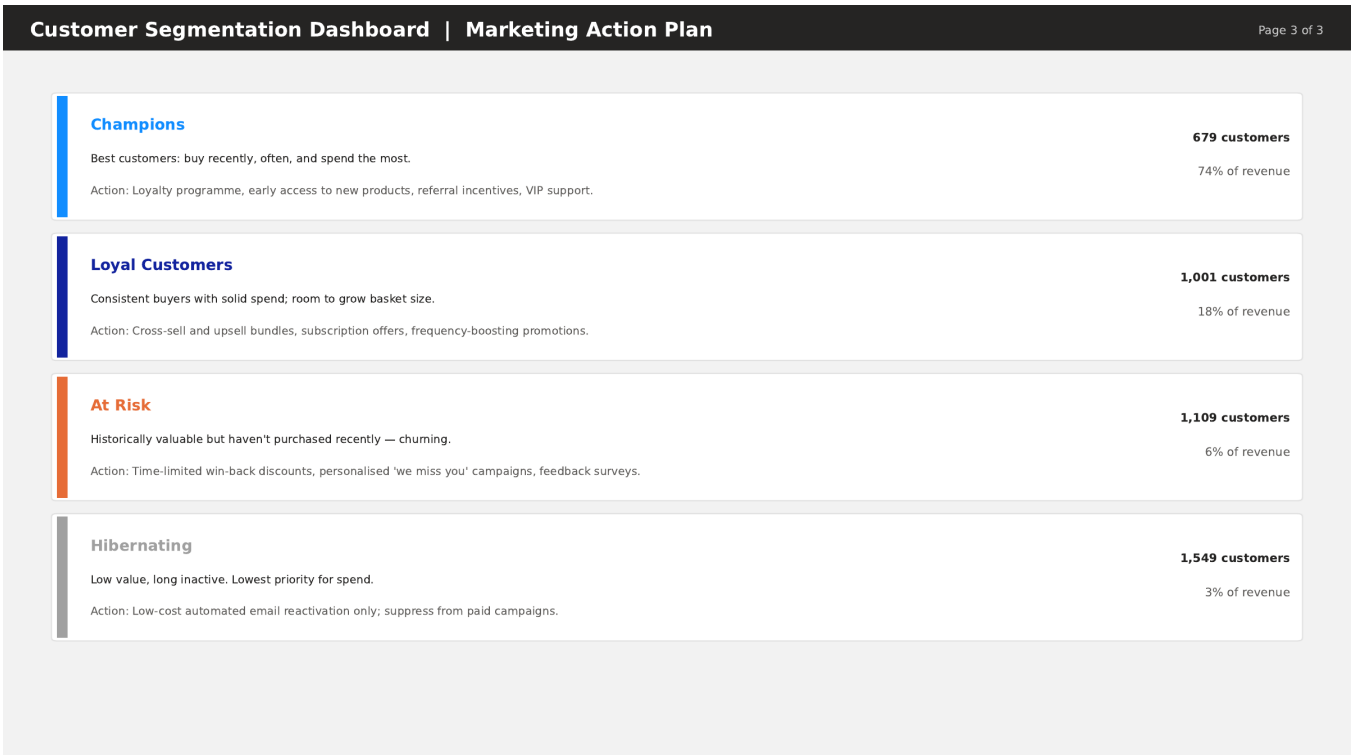

Figure 7 — Dashboard Page 3: Marketing action plan.

12. Key Findings

- **Revenue is highly concentrated.** 15.7% of customers (Champions) generate 73.8% of revenue — a textbook Pareto distribution.
- **The largest segment is the least valuable.** Hibernating customers are 35.7% of the base but only 2.5% of revenue, so reactivation spend on them should be minimal.
- **A clear win-back opportunity exists.** The At Risk segment (1,109 customers, avg recency ~94 days) were once buyers and are now lapsing — the cheapest group to save before full churn.
- **Loyal Customers are the growth lever.** 1,001 steady buyers (~£2,090 each) sit just below Champion status; nudging their frequency and basket size is the clearest path to growing the core.
- **The segments are well separated** on Frequency and Monetary, giving marketing confidence the groups are real behavioural clusters, not arbitrary cuts.

13. Marketing Action Plan per Segment

Segment	Profile	Recommended action
Champions	Recent, frequent, high spend — the core	Loyalty programme, early product access, referral incentives, VIP support. Protect at all costs.
Loyal Customers	Consistent buyers with room to grow	Cross-sell & upsell bundles, subscription offers, frequency-boosting promotions.
At Risk	Previously valuable, now lapsing	Time-limited win-back discounts, personalised ‘we miss you’ campaigns and feedback surveys — act before they churn.
Hibernating	Low value, long inactive	Low-cost automated email reactivation only; suppress from paid campaigns to protect ROI.

14. Conclusion, Limitations & Future Work

This project takes raw, messy transaction logs and turns them into a labelled customer base with a clear, budget-relevant story: a small core of Champions funds the business, a sizeable At Risk group is worth saving, and a dormant majority should not absorb expensive marketing. The RFM + K-Means approach is interpretable end-to-end, and the output is operationalised in a Power BI action plan rather than left as a notebook.

Limitations

- RFM captures *what* customers did, not *why* — it ignores product category, margin, acquisition channel and seasonality.
- K-Means assumes roughly spherical, equally-sized clusters; the log + scale step makes that assumption reasonable but not guaranteed.
- The data is a single UK retailer over one year; the exact segment sizes will not transfer to other businesses without re-fitting.
- Silhouette favoured k = 2; k = 4 is a defensible business choice but reasonable analysts could argue for a different granularity.

Future work

- Add an RFM scoring grid (quartile scores 1–4 per dimension) alongside clustering for a simple, transparent cross-check.
- Compare against other algorithms (Gaussian Mixture, DBSCAN, hierarchical) that relax the spherical-cluster assumption.
- Layer in customer lifetime value (CLV) prediction so spend can be prioritised by expected future value, not just past value.
- Schedule a monthly refresh so customers move between segments over time and campaigns can react.

Appendix — Reproducing the Analysis

```
pip install pandas numpy matplotlib seaborn scikit-learn openpyxl
# place "Online Retail.xlsx" in the project folder
python customer_segmentation_rfm_kmeans.py
# Outputs: skew + elbow/silhouette charts, segment plots, customer_segments.csv
```

E-Commerce Customer Segmentation — project report by Rahul. RFM analysis + K-Means clustering in Python, visualised in Power BI.